Práctica N°0: Medicion de Corriente con ajuste de PWM visualizado mediante puerto UART

Índice

1. Objetivos **2**
- 1.1. Objetivo general . . . 2
- 1.2. Objetivos específicos . . . 2

2. Desarrollo **2**
- 2.1. Creación del Proyecto en STM32CubeIDE . . . 2

3. Asignación de pines y configuración en STM32Cube **5**
- 3.1. Resumen de pines (didáctico) . . . 5
- 3.2. Capturas del .ioc por lados del pinout . . . 6
- 3.3. Capturas específicas por periférico . . . 7
- 3.4. Ubicación segura del código en STM32CubeIDE . . . 9

4. Estructura del archivo `main.c` **10**
- 4.1. Encabezado e inclusiones . . . 10
- 4.2. Definiciones de pines (macros) . . . 11
- 4.3. Handles de periféricos . . . 11
- 4.4. Prototipos y funciones inline auxiliares . . . 11
- 4.5. Variables de usuario y tabla de pasos PWM . . . 11
- 4.6. Función para ajustar PWM . . . 12
- 4.7. Manejador de interrupción externa (botón) . . . 12
- 4.8. Lectura SPI del sensor de corriente . . . 13
- 4.9. Redirección de printf a UART . . . 13
- 4.10. Función `main` y bucle principal . . . 13
- 4.11. Inicialización de periféricos (MX_*_Init) . . . 14

5. Carga del Programa en la Placa **14**

6. Resultados **15**

1. Objetivos

1.1. Objetivo general

Diseñar e implementar, desde cero en *STM32CubeIDE*, un sistema que controle el *duty cycle* de una señal **PWM**, lea la **corriente** mediante un sensor Hall y **visualice** las variables por **UART** usando PuTTY.

1.2. Objetivos específicos

1. **Entorno y proyecto:** Crear el proyecto en *STM32CubeIDE* y configurar reloj, pines y periféricos necesarios (TIMx para PWM, interfaz del sensor Hall, y USARTx para UART).

2. **Mapeo de pines:** Asignar pines para la salida PWM, la interfaz del sensor Hall y la UART hacia el PC.

3. **PWM:** Configurar TIMx en modo PWM con frecuencia adecuada (p. ej., $> 20\,\text{kHz}$) y rango de *duty cycle* 0 %–100 %.

4. **Lectura de corriente:** Inicializar y leer el sensor Hall; convertir los datos crudos a unidades físicas (A) aplicando el factor de escala correspondiente.

5. **Telemetría UART:** Transmitir por UART (p. ej., 115200 bps) tramas legibles en PuTTY con `timestamp`, `duty_cycle` e `I_A`.

6. **Validación:** Verificar en PuTTY que variaciones del *duty* se reflejan en la corriente, adjuntando capturas de pantalla.

2. Desarrollo

2.1. Creación del Proyecto en STM32CubeIDE

Para iniciar con la práctica es necesario crear un nuevo proyecto en el entorno **STM32CubeIDE**, el cual permite configurar los periféricos del microcontrolador y generar automáticamente el código base del proyecto.

1. Abrir el programa **STM32CubeIDE**. Una vez cargada la interfaz, seleccionar la opción **File** → **New** → **STM32 Project**.

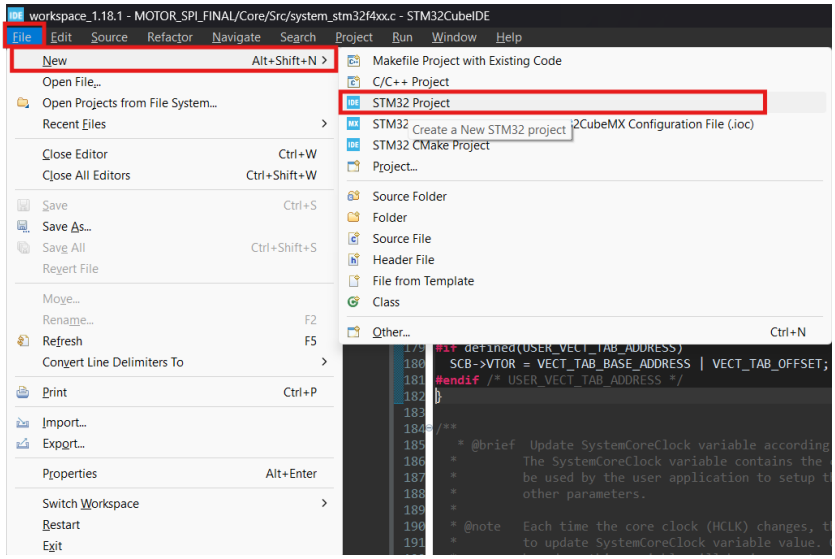


Figura 1: Selección de un nuevo proyecto en STM32CubeIDE.

2. En la ventana emergente, seleccionar la tarjeta de desarrollo utilizada. Para esta práctica se emplea la **STM32F407G-DISC1**. Se puede buscar directamente por el nombre de la tarjeta en la pestaña **Board Selector**.

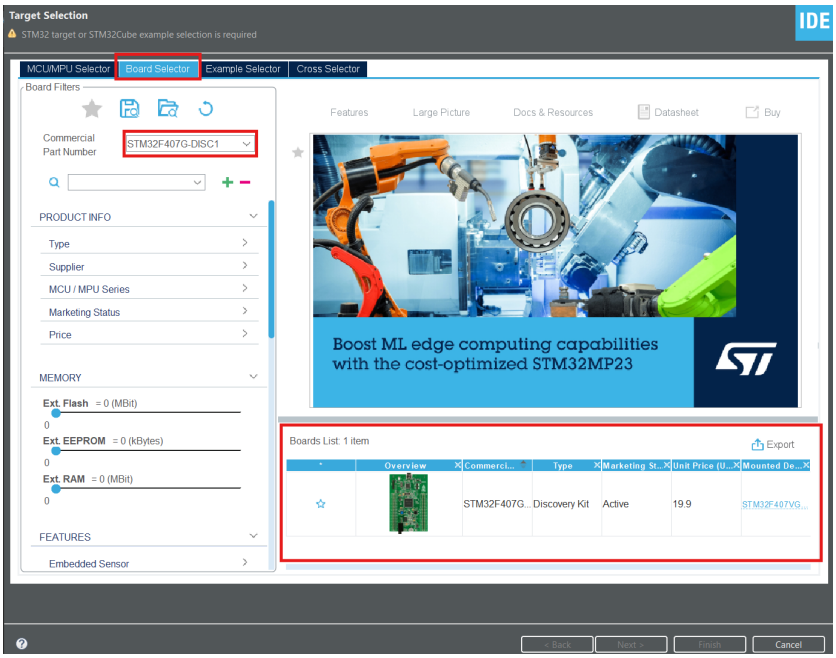


Figura 2: Selección de la tarjeta STM32F407G-DISC1 en el Board Selector.

3. Asignar un nombre al proyecto y seleccionar la carpeta donde será guardado. Confirmar con **Finish**.

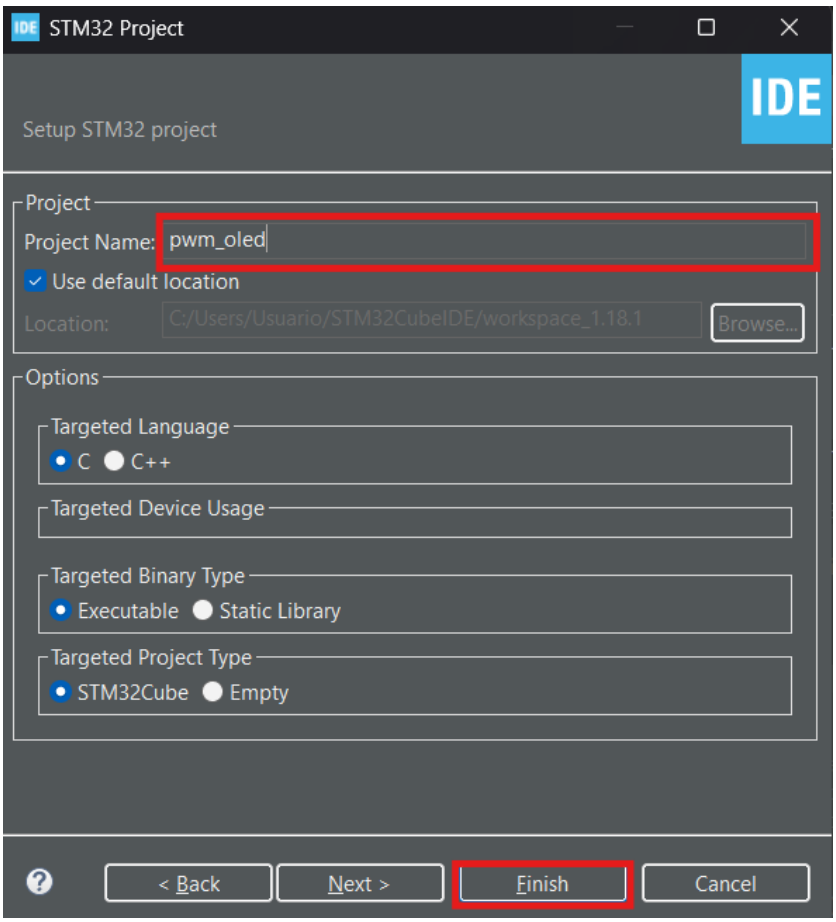


Figura 3: Asignación del nombre del proyecto.

4. Una vez creado el proyecto, se abrirá automáticamente el archivo de configuración con extensión **.ioc**, que corresponde al **STM32CubeMX** integrado dentro de CubeIDE. Este archivo permite configurar los periféricos de manera gráfica.

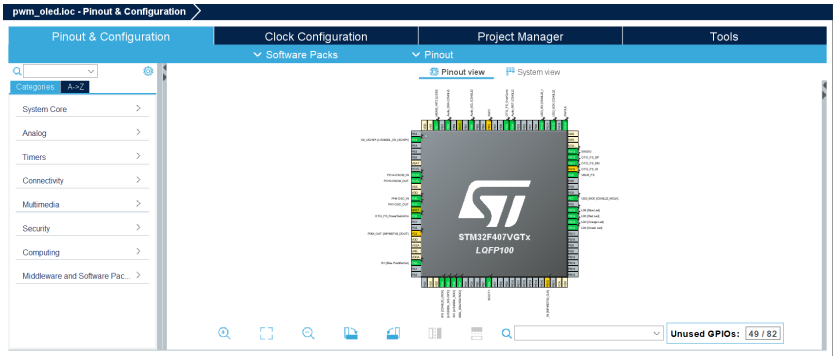


Figura 4: Interfaz inicial del archivo `.ioc`.

3. Asignación de pines y configuración en STM32Cube

3.1. Resumen de pines (didáctico)

Señal	Función	Pin MCU	Motivo
CS	SPI NSS (sensor Hall)	**PB12**	NSS de **SPI2**; elección típica para selección de esclavo.
SCK	SPI reloj	**PB13**	**SPI2_SCK**; facilita la configuración inicial.
SDO	SPI MISO	**PB14**	**SPI2_MISO**; datos del sensor hacia el MCU.
PWM	Salida PWM (TIMx_CHy)	**PE13**	Canal PWM para accionar el driver (elegir TIM/CH compatibles).
IN1	GPIO salida	**PC2**	Dirección/sentido de giro.
IN2	GPIO salida	**PE12**	Dirección/sentido de giro.
SLP	GPIO salida	**PE14**	*Sleep/enable* del driver (activo según hoja de datos).
TX	UART hacia PC	**PA2**	**USART2_TX**; se mantiene para PuTTY.
RX	UART desde PC	**PA3**	**USART2_RX**; se mantiene para *debug*.

Cuadro 1: Mapa de pines propuesto para SPI (sensor Hall), PWM (driver) y UART (telemetría).

3.2. Capturas del .ioc por lados del pinout



Figura 5: Vista del *Pinout* — **pines lado Norte**. Verificar presencia/ubicación de **PA2 (USART2_TX)** y **PA3 (USART2_RX)** según el mapeo de tu placa.

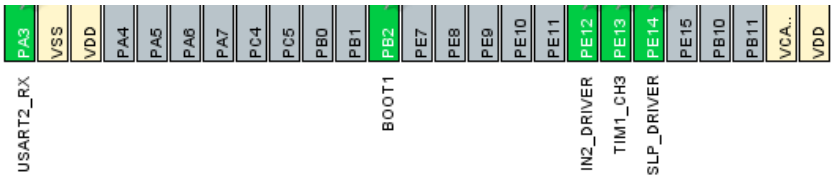


Figura 6: Vista del *Pinout* — **pines lado Sur**. Confirmar **PE12 (IN2)**, **PE13 (PWM)**, **PE14 (SLP)** en *Alternate Function/Output* según corresponda.

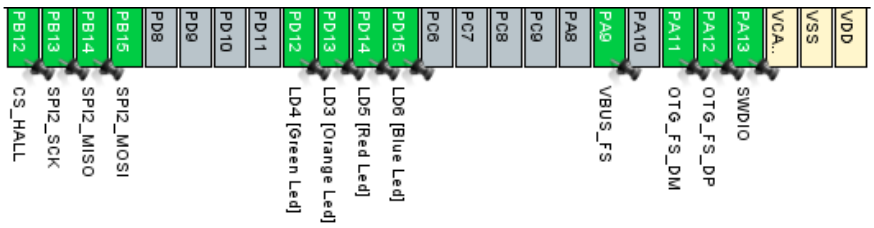


Figura 7: Vista del *Pinout* — **pines lado Este**. Revisar **PB12 (CS/NSS)**, **PB13 (SCK)**, **PB14 (SDO/MISO)** del bloque **SPI2**.

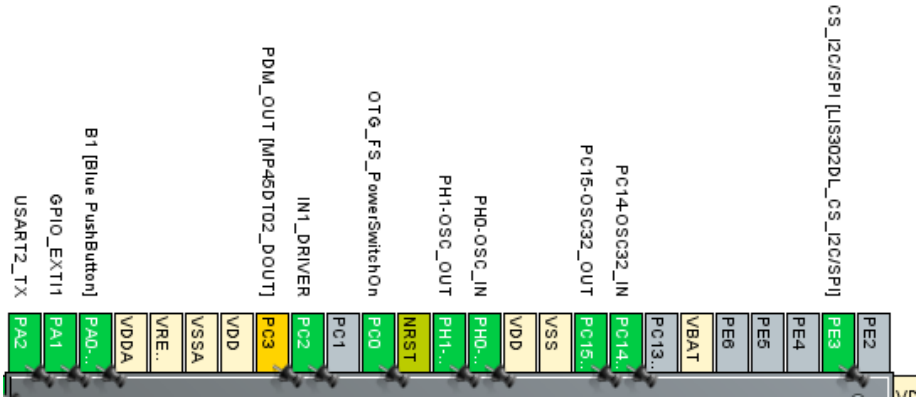


Figura 8: Vista del *Pinout* — **pines lado Oeste**.

3.3. Capturas específicas por periférico

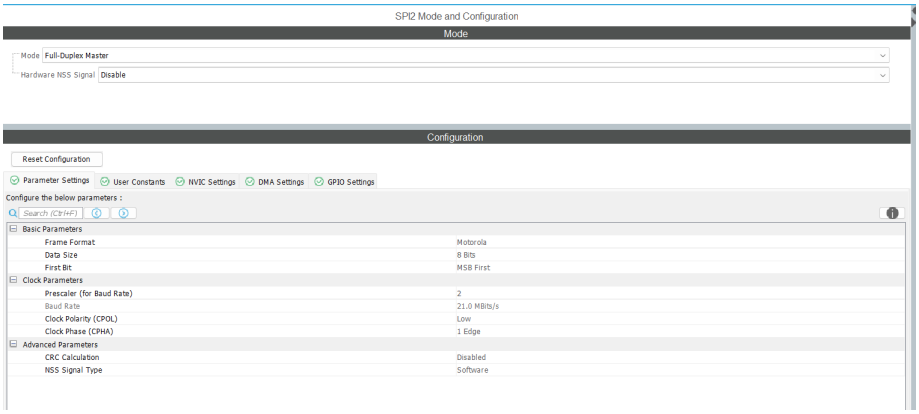


Figura 9: **SPI2** configurado con **PB12 (NSS)**, **PB13 (SCK)**, **PB14 (MISO)** .

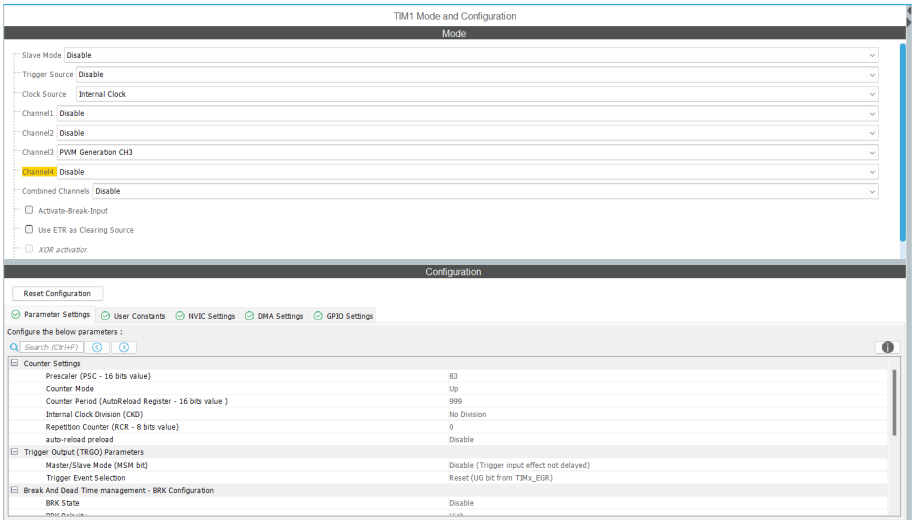


Figura 10: **PWM** en **PE13** (TIMx_CHy) .

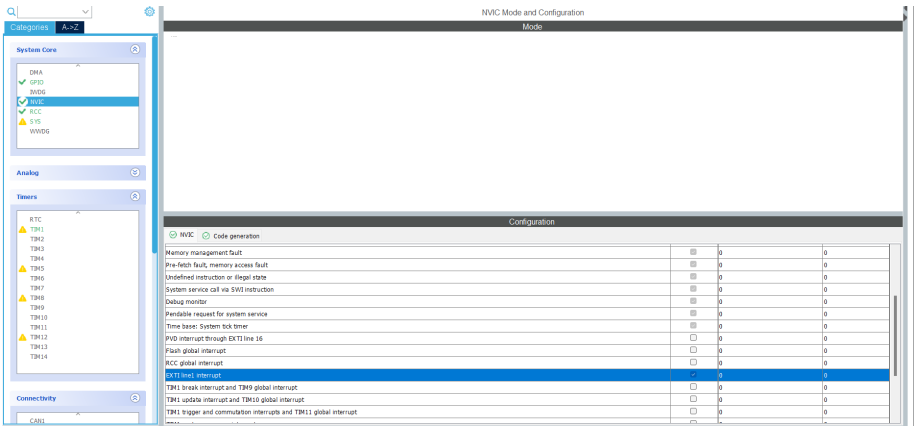


Figura 11: Habilitación de Interrupciones en el pin del pulsante.

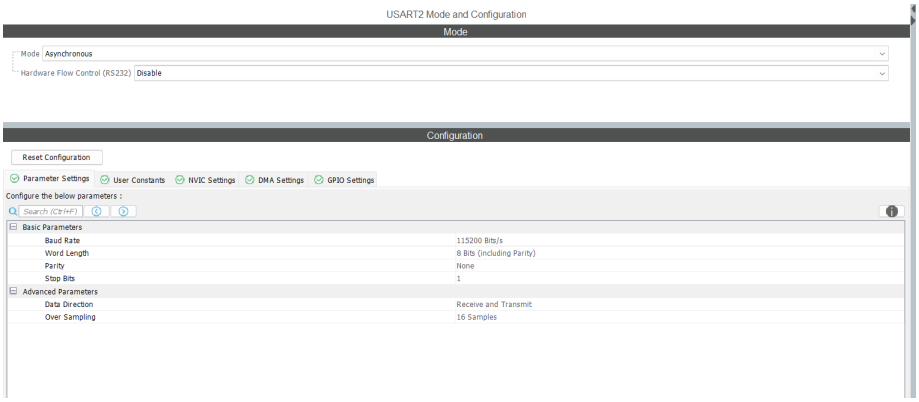


Figura 12: **USART2** en **PA2/PA3**

3.4. Ubicación segura del código en STM32CubeIDE

Al trabajar con proyectos generados desde STM32CubeMX, es importante conocer en qué partes del código se deben realizar las modificaciones para evitar que se borren cuando el proyecto se regenere.

El código que genera automáticamente STM32CubeMX incluye secciones delimitadas por comentarios especiales. Las zonas seguras para agregar código personalizado están indicadas de la siguiente manera:

```
/* USER CODE BEGIN 0 */
// Aqui se puede colocar codigo global (variables, librerias externas, etc.)
/* USER CODE END 0 */

int main(void)
{
  /* USER CODE BEGIN 1 */
  // Codigo antes de la inicializacion de perifericos
  /* USER CODE END 1 */

  /* USER CODE BEGIN 2 */
  // Codigo despues de la inicializacion de perifericos
  /* USER CODE END 2 */

  while (1)
  {
    /* USER CODE BEGIN WHILE */
    // Codigo dentro del bucle principal
    /* USER CODE END WHILE */

```

```
    /* USER CODE BEGIN 3 */
    // Codigo adicional dentro del bucle principal
    /* USER CODE END 3 */
  }
}
```

Listing 1: Secciones seguras en el archivo `main.c`

De igual forma, en los archivos de interrupciones (`stm32f4xx_it.c`) y en otros módulos del proyecto, siempre aparecerán las mismas etiquetas:

```
/* USER CODE BEGIN  */
// Codigo del estudiante
/* USER CODE END  */
```

Listing 2: Ejemplo de secciones seguras en interrupciones

Regla importante:

- Todo el código del estudiante (funciones, variables, configuraciones adicionales) debe ir únicamente dentro de las secciones delimitadas por `USER CODE BEGIN` y `USER CODE END`.
- Nunca se debe modificar el código fuera de esas áreas, ya que será sobrescrito automáticamente cuando se regenere el proyecto desde STM32CubeMX.

Esto garantiza que los cambios del estudiante permanezcan intactos incluso si se reconfiguran los periféricos o se actualiza el archivo `.ioc`.

4. Estructura del archivo `main.c`

A continuación se presenta una explicación por secciones del fichero `main.c` suministrado. Está pensada para un estudiante que se inicia en STM32CubeIDE/STM32CubeMX: cada bloque muestra el código relevante dentro de un entorno `lstlisting` y una explicación clara y concisa.

4.1. Encabezado e inclusiones

```
/* Includes --*/
#include "main.h"
#include "usb_host.h"
#include <stdio.h>
```

Explicación: `main.h` contiene las definiciones generadas por CubeMX (pines, prototipos). `usb_host.h` es el middleware para el host USB. `stdio.h` se incluye para usar `printf` redirigido vía UART (ver `__io_putchar`).

4.2. Definiciones de pines (macros)

```
#define CS_HALL_Pin         GPIO_PIN_12
#define CS_HALL_GPIO_Port   GPIOB
#define IN1_DRIVER_Pin         GPIO_PIN_2
#define IN1_DRIVER_GPIO_Port   GPIOC
#define IN2_DRIVER_Pin         GPIO_PIN_12
#define IN2_DRIVER_GPIO_Port   GPIOE
#define SLP_DRIVER_Pin         GPIO_PIN_14
#define SLP_DRIVER_GPIO_Port   GPIOE
```

Explicación: CubeMX genera macros para acceder a pines por nombre legible. Usar estas macros evita escribir directamente números de pin/puerto y hace el código más mantenible.

4.3. Handles de periféricos

```
SPI_HandleTypeDef hspi2;
TIM_HandleTypeDef htim1;
UART_HandleTypeDef huart2;
```

Estos objetos ("handles") mantienen el estado y la configuración de cada periférico (SPI2, TIM1, USART2). Son inicializados por las funciones `MX_*_Init()` generadas por CubeMX.

4.4. Prototipos y funciones inline auxiliares

```
static inline void driver_enable(uint8_t en) {
HAL_GPIO_WritePin(SLP_DRIVER_GPIO_Port, SLP_DRIVER_Pin,
   en ? GPIO_PIN_SET : GPIO_PIN_RESET);
}

typedef enum { DIR_ADELANTE = 0, DIR_ATRAS = 1 } dir_t;
static inline void driver_set_dir(dir_t d) { ... }
```

Explicación: funciones `inline` pequeñas para controlar el driver (activar/desactivar sleep y seleccionar dirección). Usar `HAL_GPIO_WritePin` es la forma HAL de cambiar pines.

4.5. Variables de usuario y tabla de pasos PWM

```
uint32_t last_press_time = 0;  // Antirrebote
volatile uint8_t step_count = 0;
const uint8_t duty_steps[6] = {0, 20, 40, 60, 80, 90};
```

`last_press_time` guarda el tiempo del último pulso para debouncing. `step_count` es `volatile` porque puede cambiar en una ISR. `duty_steps` define los porcentajes de duty cycle predefinidos.

4.6. Función para ajustar PWM

```
void Set_PWM_Duty(uint8_t percent) {
if (percent > 100) return;
uint32_t pulse = (percent * (__HAL_TIM_GET_AUTORELOAD(&
   htim1) + 1)) / 100;
__HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, pulse);
}
```

Explicación: calcula el valor del çompare.ª partir del autoreload del temporizador (ARR) para obtener el duty en porcentaje. Esto permite cambiar el ciclo de trabajo sin tocar la configuración del temporizador.

4.7. Manejador de interrupción externa (botón)

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
if (GPIO_Pin == GPIO_PIN_1) {
if (HAL_GetTick() - last_press_time > 50) {
step_count = (step_count + 1) % 6;
Set_PWM_Duty(duty_steps[step_count]);
last_press_time = HAL_GetTick();
}
}
}
```

Explicación: esta es la çallback"HAL para interrupciones por GPIO (configurada en `MX_GPIO_Init`). Usa `HAL_GetTick()` para antirrebote y avanza al siguiente paso de duty cycle.

4.8. Lectura SPI del sensor de corriente

```
uint16_t Read_Current_Raw(void) {
uint8_t tx_dummy[2] = {0x00, 0x00};
uint8_t rx_data[2] = {0};
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_12, GPIO_PIN_RESET);
   // CS low
HAL_SPI_TransmitReceive(&hspi2, tx_dummy, rx_data, 2,
   HAL_MAX_DELAY);
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_12, GPIO_PIN_SET); //
   CS high
uint16_t raw = ((uint16_t)rx_data[0] << 8) | rx_data[1];
return raw & 0x0FFF;  // 12 bits
}
```

Explicación: protocolo SPI típico: bajar CS, intercambiar bytes, subir CS. El valor leído se empaqueta y se limita a 12 bits.

4.9. Redirección de printf a UART

```
int __io_putchar(int ch) {
HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1,
   HAL_MAX_DELAY);
return ch;
}
```

Nota: esta función permite usar `printf` en el firmware (depende del retargeting de la librería estándar en su proyecto).

4.10. Función `main` y bucle principal

```
int main(void) {
HAL_Init();
SystemClock_Config();
MX_GPIO_Init();
MX_USB_HOST_Init();
MX_SPI2_Init();
MX_TIM1_Init();
MX_USART2_UART_Init();
driver_enable(1);
driver_set_dir(DIR_ADELANTE);
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_3);
```

```
Set_PWM_Duty(0);
while (1) {
MX_USB_HOST_Process();
// toma 10 muestras espaciadas y calcula promedio
// calcula corriente en mA y la imprime
// aumenta PWM si la corriente excede un umbral
HAL_Delay(1000);
}
}
```

Explicación: el flujo típico generado por CubeMX: inicializar HAL, configurar reloj, inicializar periféricos (las funciones `MX_*_Init` son generadas automáticamente). Después viene la inicialización específica del usuario (activar driver, iniciar PWM) y el bucle infinito donde se leen sensores, se procesan datos y se toman decisiones.

4.11. Inicialización de periféricos (MX_*_Init)

```
static void MX_TIM1_Init(void) { ... }
static void MX_SPI2_Init(void) { ... }
static void MX_USART2_UART_Init(void) { ... }
static void MX_GPIO_Init(void) { ... }
```

Estos bloques contienen la configuración detallada (prescaler, periodo, polaridades, modos SPI/UART, configuración de pines). Si modificas parámetros en CubeMX, estas funciones son reescritas, por eso las secciones de usuario están delimitadas con `/* USER CODE BEGIN */` / `END` para preservar cambios manuales.

5. Carga del Programa en la Placa

Una vez configurados los periféricos y desarrollado el código fuente, el siguiente paso es **cargar el programa en la placa STM32F407G-DISC1**. Para ello, el entorno de desarrollo **STM32CubeIDE** ofrece un flujo de compilación, depuración y carga que debe seguirse de manera ordenada:

1. **Compilación del proyecto**: se realiza utilizando el icono del **martillo** ubicado en la barra de herramientas. Este paso genera los archivos binarios y permite verificar si existen errores de sintaxis o configuración.

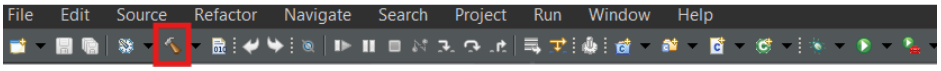


Figura 13: Icono de Compilación del proyecto mediante el icono del martillo.

2. **Depuración inicial (Debug Communication)**: en este paso se utiliza el icono con forma de **cucaracha**, que inicia la depuración y establece la comunicación con la placa. Aquí se pueden detectar errores relacionados con la conexión o la carga del firmware.

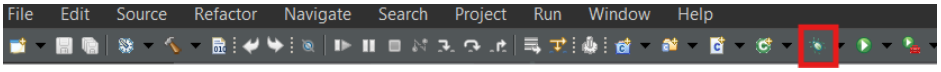


Figura 14: Iconon de Depuración inicial con el icono de la cucaracha en STM32CubeIDE.

3. **Carga final del programa**: si no se presentan errores en los pasos anteriores, el programa es transferido a la memoria del microcontrolador. Una vez cargado, la placa ejecutará automáticamente el código y se podrán observar los resultados directamente en el hardware.

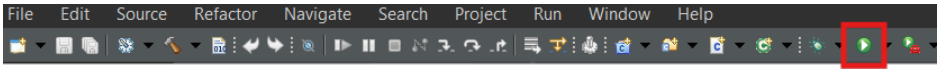


Figura 15: Icono de carga del programa en la placa STM32F407G-DISC1.

6. Resultados

Durante la ejecución del programa, el sistema permitió el control del motor mediante el ajuste del ciclo útil de la señal PWM y la visualización simultánea de la corriente medida en la pantalla OLED. En la Figura 17 se muestra una captura de pantalla del funcionamiento correcto del sistema, donde se observa la lectura de la corriente proveniente del sensor TLI4970 y la variación del valor al modificar el ciclo útil con los pulsadores.

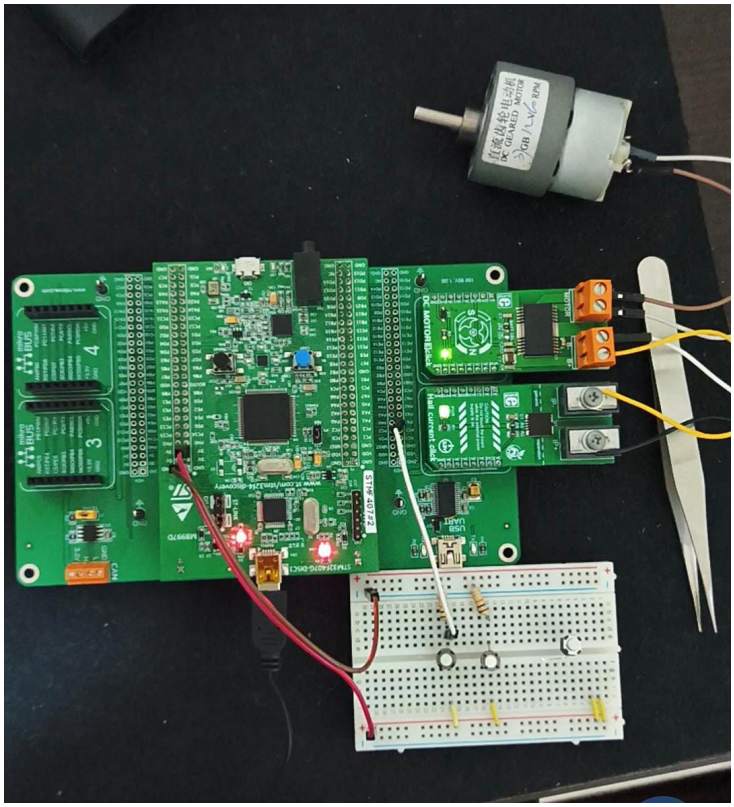


Figura 16: Funcionamiento del sistema mostrando la lectura de corriente y el control PWM.

El sistema responde de forma estable y coherente con los cambios de control, validando la correcta integración de los periféricos SPI, PWM, UART y las interrupciones externas implementadas en el archivo `main.c`.

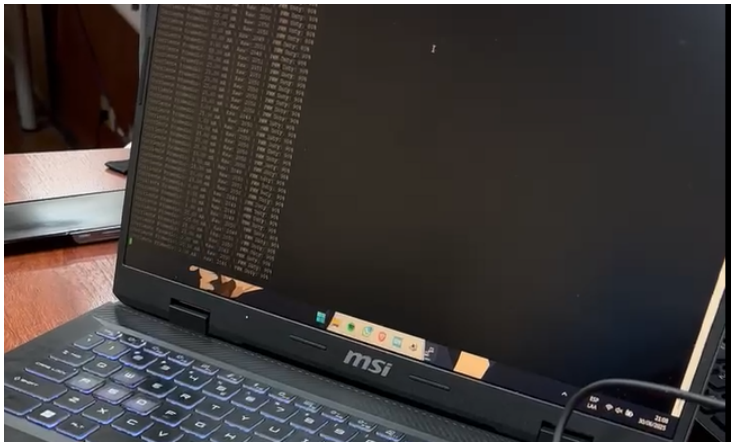


Figura 17: Funcionamiento del sistema mostrando la lectura de corriente y el control PWM.